

SKILL GAP ANALYSIS & LEARNING RECOMMENDATIONS

Project submitted to the
SRM University – AP, Andhra Pradesh
for the partial fulfillment of the requirements to award the degree of

Bachelor of Technology

In

Computer Science and Engineering

School of Engineering and Sciences

Submitted by

Afthab Md AP23110010398

Akhil J AP23110010037

Srihaas N AP23110011113

Dhanush K AP21110010487

Avinash K AP23110010144



Under the Guidance of

Mr. Himanshu Mishra

Assistant professor, Dept.of CSE

SRM University–AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240

April, 2026

CERTIFICATE

Date: 27-04-2026

This is to certify that the work present in this Project entitled “**SKILL GAP ANALYSIS & LEARNING RECOMMENDATIONS** ” has been carried out by J Akhil Joy, Md Afthab Raza Hussain, , N Srihaas, K Dhanush, S Avinash under my supervision. The work is genuine, original, and suitable for submission to the SRM University – AP for the award of Bachelor of Technology in School of Engineering and Sciences.

Supervisor

(Signature)

Mr. Himanshu Mishra

Assistant Professor,

Department of CSE

Acknowledgements

We would like to acknowledge the guidance and support provided by our mentor and advisor **Mr. Himanshu Mishra**. Your wisdom and experience helped steer us in the right direction, and your feedback was instrumental in refining our ideas and approaches.

I extend my heartfelt thanks to my team members who collaborated tirelessly, sharing their expertise and insights, and working together cohesively towards our common goal. Each of you played a crucial role in shaping the project's outcome, and I am truly grateful for your contributions.

Furthermore, I am thankful for the resources and opportunities provided by our university, which enabled us to carry out this project successfully.

Abstract

The contemporary job market is characterised by rapid technological evolution, making it increasingly difficult for students and fresh graduates to remain aligned with current industry expectations. The Skill Gap Analysis and Learning Recommendation System is a full-stack web application developed to address this challenge by providing students with a data-driven, personalised framework for career preparation. The system allows users to define their target career roles, evaluate their existing competencies through automated resume parsing and self-assessment, and receive a prioritised, structured learning roadmap that closes the identified skill gap.

Built on the MERN (MongoDB, Express.js, React.js, Node.js) technology stack, the platform integrates resume parsing, dynamic gap analysis, and real-time progress tracking within a unified interface. The readiness score engine calculates a percentage-based compatibility metric between a user's current skill profile and the industry-mandated requirements for a selected role. Administrators of the platform are empowered to manage the knowledge base of roles, skills, and resources, thereby ensuring the system remains updated in line with evolving industry trends.

The application incorporates gamification elements such as learning streaks and animated progress indicators to sustain user motivation. By transforming complex career development data into clear, actionable insights, this system serves as a digital mentor for thousands of students seeking direction in their professional journeys. This report comprehensively documents the design, architecture, implementation, and deployment of the system, serving as a complete technical reference for evaluation and future development.

1. INTRODUCTION

In an era defined by technological disruption and constant industrial evolution, the ability to align one's academic preparation with real-world professional expectations has become a defining factor for career success. The global job market no longer evaluates candidates purely on the basis of formal academic qualifications; instead, it places enormous emphasis on demonstrable, role-specific technical and soft skills. This shift has created a formidable challenge for students and fresh graduates who, despite years of rigorous academic training, frequently find themselves ill-equipped to meet the precise expectations of their desired employers.

The Skill Gap Analysis and Learning Recommendation System is a comprehensive full-stack web application engineered to bridge this divide. The system is built around the core premise that students require not merely information but structured, personalized guidance. By enabling users to select a target job role - such as Full Stack Developer, Data Scientist, or DevOps Engineer, the platform performs an intelligent comparison between the skills a student currently possesses and the skills that the industry demands for that specific role. The output of this comparison is a quantitative Readiness Score, expressed as a percentage, accompanied by a clearly prioritized list of Skill Gaps that must be addressed.

The significance of this project cannot be overstated in today's context. According to industry surveys, a substantial portion of engineering graduates report feeling underprepared for their first professional role despite achieving strong academic records. This disconnect arises because university curricula, while theoretically rigorous, often lag behind the pace of industry adoption of new frameworks, tools, and methodologies. A student may graduate with strong knowledge of data structures and algorithms but have no exposure to containerization tools like Docker or distributed systems frameworks like Apache Kafka, both of which are prerequisites for modern backend engineering roles.

The system also recognizes that guidance alone is insufficient without accountability. To address this, the platform incorporates a streak-based motivation engine that tracks daily learning activity and visual progress charts that update in real time as users complete learning modules. The result is an experience that is simultaneously diagnostic, prescriptive, and motivational — transforming the often overwhelming process of career preparation into a manageable, measurable journey.

2. SCENARIO BASED CASE STUDY

To appreciate the practical value and real-world applicability of the Skill Gap Analysis and Learning Recommendation System, it is instructive to examine the journey of a representative user — a final-year Computer Science student named Rahul Sharma — who aspires to secure a position as a Full Stack Web Developer at a reputed technology company upon graduation.

2.1 Background and Initial Challenge

Rahul is a final-year B.E. student with a cumulative GPA of 8.4 out of 10. He is diligent, academically motivated, and technologically curious. Despite his strong academic performance, Rahul is acutely aware that understanding concepts in a classroom setting does not automatically translate into industry readiness. He has heard from seniors that companies like Infosys, Wipro, and various product-based startups require candidates for Full Stack Developer roles to demonstrate proficiency in React.js, Node.js, RESTful API design, MongoDB, Git, and cloud deployment platforms such as AWS or Heroku. Rahul is uncertain about how many of these competencies he possesses at the required depth. He lacks a structured method to assess his readiness and determine what to learn next.

2.2 Onboarding and Initial Assessment

Rahul discovers the platform through his college's placement preparation portal and completes the registration process. After verifying his email and logging in, he is directed to the Career Wizard — a guided onboarding module designed to collect essential information. He uploads his most recent resume in PDF format and selects 'Full Stack Web Developer' as his primary target career role.

The backend processing engine immediately invokes the resume parsing module, which uses the pdf-parse library to extract raw text from the uploaded file. This text is tokenized and passed through a pattern-matching algorithm that cross-references the extracted content against the system's comprehensive skill database. The algorithm identifies mentions of C++, basic HTML, Microsoft Office, and Java as Rahul's current skill set. It records these findings against his user profile in the database as the baseline for subsequent analysis.

2.3 Gap Analysis and Readiness Score

The Gap Analysis Engine then retrieves the skill requirements associated with the 'Full Stack Web Developer' role from the database. These requirements include competencies such as React.js (proficiency level 80/100), Node.js and Express.js (75/100), MongoDB (70/100), RESTful API design (75/100), Git version control (80/100), and introductory knowledge of cloud services such as AWS (60/100). The engine compares the required proficiencies against Rahul's currently identified skill levels. Given that his profile reflects only surface-level familiarity with HTML and no exposure to the remaining required technologies, the computed Readiness Score is 35%.

The dashboard presents this result visually through an animated readiness gauge rendered using Chart.js. A colour-coded gap summary lists each missing or underdeveloped skill, ranked by importance. React.js and Node.js are flagged as 'High Priority,' while cloud deployment is marked as 'Medium Priority,' and basic development environment setup is marked as 'Low Priority.' This clear stratification prevents Rahul from feeling overwhelmed and helps him understand the sequence in which to address his learning goals.

2.4 Learning Journey and Progress Tracking

Acting on the system's recommendation, Rahul begins with the React.js learning module. The platform provides him with a curated list of resources, including links to the official React documentation, beginner-to-intermediate video tutorials, and hands-on project prompts. As he completes each resource and marks it as done, the system increments his proficiency score for React.js, updates his Readiness Score in real time, and maintains his learning streak counter. Within the first two weeks, his overall Readiness Score climbs from 35% to 52%.

2.5 Before vs. After Impact

By the conclusion of the semester, Rahul has methodically worked through the learning paths for React.js, Node.js, MongoDB, and Git. His Readiness Score has increased to 82%, reflecting a near-complete alignment with the requirements of his target role. More significantly, he now possesses a verifiable portfolio of completed learning activities and mini-projects that he can present confidently during campus recruitment. The system has transformed Rahul from a capable but directionless student into a focused, industry-ready professional candidate — demonstrating the tangible value that the platform delivers to its users.

3. SYSTEM REQUIREMENTS

3.1 Software Requirements

The technology stack selected for this project has been carefully evaluated and chosen on the basis of performance, community support, scalability, and fitness for purpose within the context of a career-oriented web application. Each technology addresses a specific layer of the system, ensuring that the overall architecture is both modular and maintainable.

React.js and Vite (Frontend Framework and Build Tool): React.js was selected as the primary frontend framework due to its component-based architecture, which is ideally suited for applications that require a high degree of interactivity and real-time data updates. In this system, numerous UI components — such as the readiness gauge, skill cards, and progress bars — need to re-render dynamically in response to user actions. React's virtual DOM mechanism ensures that only the components directly affected by a state change are updated, resulting in optimal rendering performance. Vite is employed as the build toolchain because of its native ES module-based development server.

Node.js and Express.js (Backend Runtime and API Framework): Node.js provides a non-blocking, event-driven runtime environment that is particularly well-suited for I/O-intensive applications such as this one, which handles frequent database queries, resume parsing operations, and API responses. Express.js is layered on top of Node.js as the web application framework, offering a minimal yet powerful abstraction for defining RESTful API routes, applying middleware, and managing request-response cycles. The combination enables the backend to serve multiple concurrent user requests efficiently without spawning additional threads, a model that scales well for the expected user load of a college-level deployment.

MongoDB and Mongoose (Database and ODM): MongoDB is a NoSQL document-oriented database chosen for its exceptional flexibility in handling semi-structured data. User skill profiles, career roadmaps, and learning progress records are inherently variable in structure — different users pursuing different roles will have widely differing data configurations. MongoDB's schema-less document model accommodates this variability without the rigidity of relational database schemas. Mongoose is integrated as the Object Data Modeling library, providing schema definition, data validation, relationship management through ObjectId references, and pre-save hooks for operations such as password hashing. Together, these tools provide a robust and scalable data persistence layer.

JWT and Bcrypt.js (Authentication and Security): Secure user authentication is implemented using JSON Web Tokens (JWT). Upon a successful login, the server issues a signed token containing the user's identity and role. This token is attached to subsequent API requests by the client, allowing the server to validate the user's identity without maintaining session state — a stateless approach that is both scalable and suitable for RESTful architectures. Bcrypt.js complements JWT by handling password security; all user passwords are hashed using the bcrypt algorithm with a configurable salt factor before storage, ensuring that raw credentials are never persisted in the database.

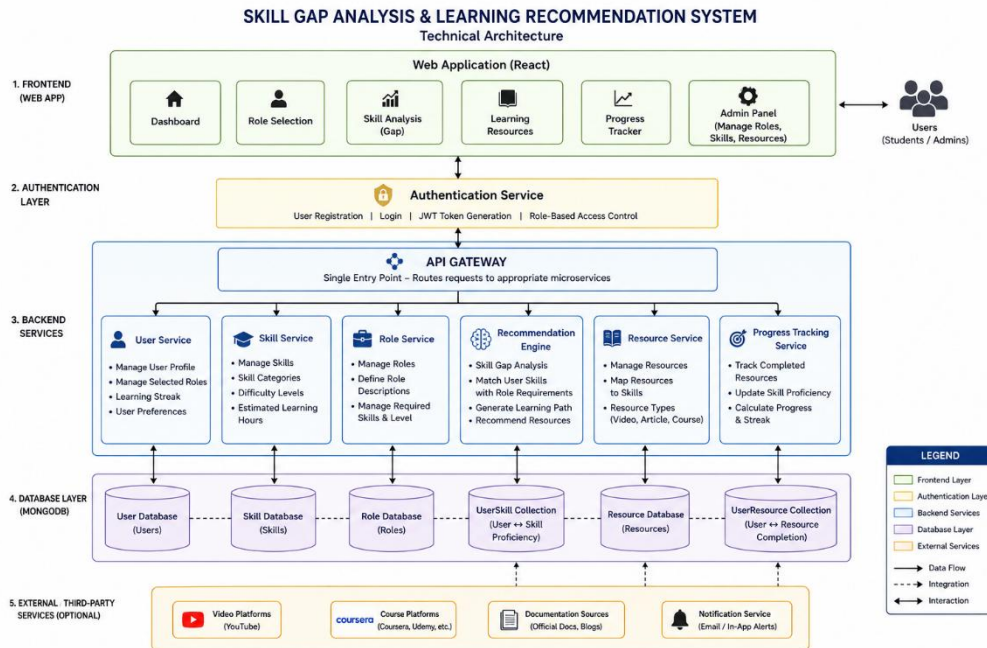
Chart.js (Data Visualization): Chart.js is integrated into the frontend to render dynamic, interactive visualizations of user progress data. It provides the animated readiness dial, skill proficiency bar charts, and historical learning activity graphs. Chart.js is preferred for this application due to its lightweight footprint, extensive customization options, and native compatibility with React via wrapper libraries.

3.2 Hardware Requirements

The system has been designed to be accessible on standard consumer-grade hardware, ensuring broad usability across diverse student populations. The minimum hardware configuration required to operate the development environment includes a dual-core processor such as the Intel Core i3 or its AMD equivalent, which provides sufficient computational capacity for running the frontend development server and the Node.js backend concurrently. A quad-core processor such as the Intel Core i5 is recommended for a smoother development experience, particularly during build processes and PDF parsing operations, which are computationally intensive tasks.

A minimum of 4 GB of system RAM is required to run both the client and server processes simultaneously without performance degradation. However, 8 GB or more is strongly recommended, as PDF text extraction and React's hot module replacement can consume significant memory during active development. For storage, a minimum of 500 MB of available disk space is necessary to accommodate the project source files, Node.js module dependencies, and local build artifacts. A solid-state drive is preferred over a traditional hard disk drive to capitalize on faster read-write speeds, which reduce build times and improve the responsiveness of file-watching operations. An active internet connection is essential for connecting to MongoDB Atlas, accessing external learning resources, and pulling dependencies during installation.

4. PROJECT ARCHITECTURE



The architectural design of the Skill Gap Analysis and Learning Recommendation System follows the well-established three-tier client-server model, which cleanly separates the system into three distinct layers: the Presentation Layer, the Application Layer, and the Data Layer. This separation of concerns promotes modularity, facilitates independent testing and deployment of individual components, and simplifies long-term maintenance.

4.1 Presentation Layer (Frontend)

The Presentation Layer is implemented as a React.js single-page application (SPA) built with Vite. This layer is responsible for rendering the user interface, capturing user interactions, and communicating with the backend through asynchronous HTTP requests made via the Axios library. When a user clicks a button such as 'Analyze My Gap,' the React component responsible for that interaction composes an HTTP POST request containing the relevant data — such as the selected role ID and any self-reported skill updates — and dispatches it to the backend API. State management within the frontend is handled using React's built-in `useState` and `useEffect` hooks, ensuring that the UI reflects the latest data at all times. The Framer Motion library augments the interface with fluid animations and page transitions, contributing to a polished user experience.

4.2 Application Layer (Backend)

The Application Layer is built on Node.js and Express.js and serves as the intelligence centre of the system. It receives requests from the frontend, applies business logic, interacts with the database, and returns structured JSON responses. All protected API routes are guarded by an authentication middleware module that intercepts incoming requests, extracts the JWT from the Authorization header, verifies its integrity and expiration using the jsonwebtoken library, and either permits the request to proceed or returns a 401 Unauthorized response. This middleware runs silently on every protected route, ensuring that unauthenticated or impersonated requests are never processed.

The core gap analysis logic resides in the Analysis Controller. When the gap analysis endpoint (POST /api/v1/analysis/gap) is invoked, the controller retrieves the role requirements and the user's skill profile from MongoDB using Mongoose queries with population.

It then iterates through each required skill, compares the required proficiency level against the user's recorded proficiency, and computes a weighted readiness score. The result — including the overall percentage, a list of identified gaps, and a prioritized learning path — is serialized into a JSON response and returned to the frontend.

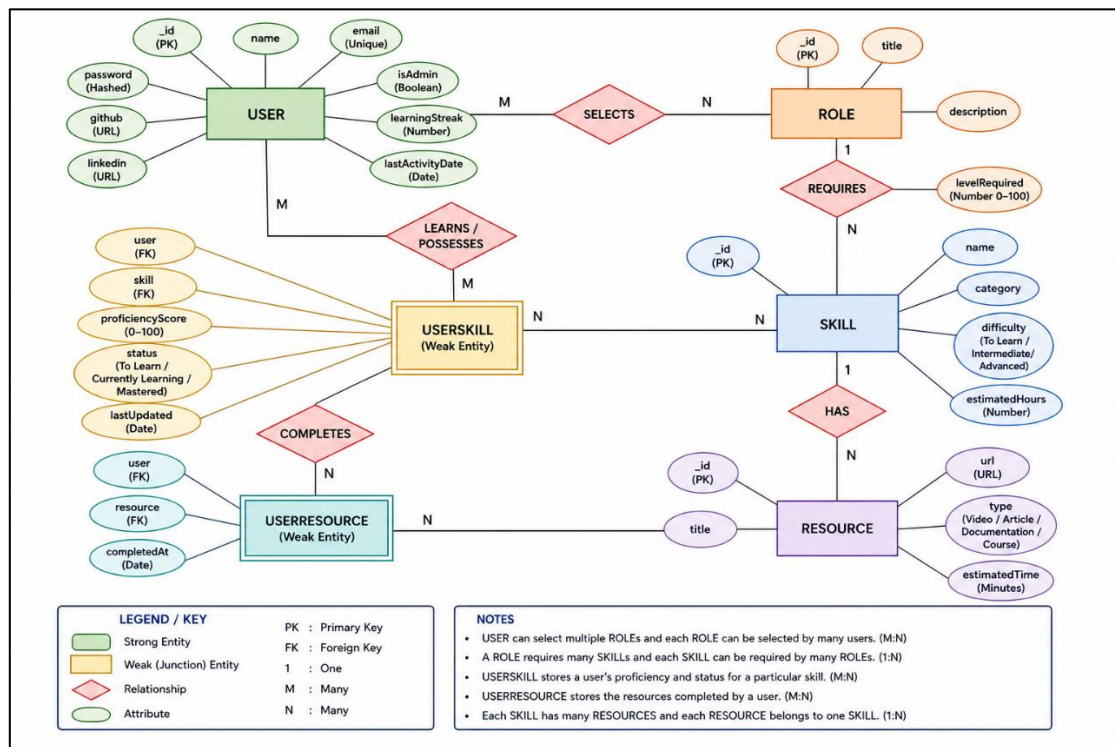
4.3 Data Layer (Database)

The Data Layer consists of a MongoDB database, managed either as a local installation or through MongoDB Atlas for cloud-based deployments. All persistent application data — including user profiles, role definitions, skill records, learning resources, and user progress entries — is stored in this layer. Mongoose schemas define the structure and validation rules for each collection, and ObjectId references enforce relational constraints between documents. For example, a UserSkill document maintains a reference to both the User and the Skill it pertains to, enabling the backend to efficiently fetch a complete picture of a user's skill inventory using Mongoose's populate mechanism. The database is accessed through a connection pool managed by Mongoose's underlying MongoDB driver, ensuring that concurrent user sessions are served without connection bottlenecks.

4.4 Security and Request Cycle Summary

Every complete request-response cycle is secured from initiation to completion. The client includes the JWT in the request header; the middleware validates it; the controller processes the business logic; the database stores or retrieves the necessary data; and the server returns a structured response that the frontend renders for the user. This end-to-end security model, combined with the layered architecture, ensures both system integrity and a seamless user experience.

5. ER DIAGRAM



5.1 Primary Entities

User: The User entity represents every individual who registers on the platform, whether as a student or an administrator. It stores personal identification data including full name, email address (which serves as a unique identifier for authentication), and a hashed password. It also maintains profile information such as social links to GitHub and LinkedIn profiles, the user's selected target roles, and activity metadata including the current learning streak count and the date of last activity. An administrative flag within the User entity distinguishes between standard student accounts and elevated administrator accounts, enabling role-based access control throughout the application.

Role: The Role entity represents a career path available for analysis within the system, such as 'Frontend Developer,' 'Data Scientist,' or 'DevOps Engineer.' Each role document includes a descriptive title, a brief industry-facing description, and an embedded array of Skill references, each annotated with a requiredLevel attribute that specifies the minimum proficiency score a user must achieve for that skill to be considered role-ready. The Role entity is primarily maintained by administrators and constitutes the 'industry standard' benchmark against which user profiles are measured.

Skill: The Skill entity represents a discrete, learnable competency relevant to one or more career roles. Attributes include the skill's name (e.g., 'React.js'), category (e.g., 'Frontend,' 'Backend,' 'Database'), difficulty level (Beginner, Intermediate, or Advanced), and an estimated number of learning hours required to achieve proficiency. The Skill entity functions as a centralized knowledge base entry that is referenced by both the Role entity and the UserSkill junction entity.

Resource: The Resource entity represents a specific learning material associated with a skill. It stores the resource title, type (Video, Article, Documentation, or Course), URL, and an estimated completion time. Resources are curated by administrators and linked directly to individual skills, providing users with structured, high-quality materials to support their learning journey.

5.2 Junction Entities and Relationships

UserSkill: The UserSkill junction entity manages the many-to-many relationship between the User entity and the Skill entity. Each UserSkill document captures a specific user's proficiency in a specific skill at a given point in time. It maintains a proficiency score (0–100), a status indicator (To Learn, Currently Learning, or Mastered), and a timestamp recording when the skill was last updated. Separating a user's proficiency data from the global Skill definition is a critical design decision; it allows administrators to update a skill's global description or difficulty rating without inadvertently affecting the progress records of any individual user.

UserResource: The UserResource junction entity tracks which specific learning resources an individual user has completed. It stores the user ID, the resource ID, and a completion timestamp. This granular tracking enables the system to display accurate completion percentages for individual learning modules and informs the streak calculation engine.

Role-Skill Association: The Role entity embeds an array of skill references, each paired with a levelRequired attribute. This design choice — embedding the relationship within the Role document rather than creating a separate junction collection — optimizes read performance when retrieving role requirements, as the full set of required skills and their thresholds can be fetched in a single database query.

5.3 Keys and Identifiers

Every document in the MongoDB database is assigned a unique ObjectId as its primary key by default, denoted as `_id`. This ObjectId is a 12-byte BSON value that is globally unique across the database and serves as the basis for all cross-collection references. In Mongoose, foreign key-style references are implemented using the type: `mongoose.Schema.ObjectId` data type within schema definitions, allowing the `populate` function to resolve references into full document objects during query execution. The User entity additionally enforces uniqueness on the email field, providing a human-readable unique identifier for authentication purposes.

6. KEY FEATURES

The Skill Gap Analysis and Learning Recommendation System is distinguished from generic learning management platforms by a set of carefully designed, purposeful features. Each feature has been conceived with a specific user pain point in mind and engineered to deliver measurable value to the student throughout their career preparation journey.

6.1 Career Wizard and Resume Parser

The Career Wizard is the first experience a new user encounters after registration. It is a structured, multi-step onboarding process that guides the user through defining their career aspirations and establishing their baseline skill profile. The cornerstone of this wizard is the resume parsing module, which allows users to upload their existing resume in PDF format. The backend leverages the pdf-parse library to extract unstructured text from the uploaded file and subsequently applies a pattern-matching algorithm to identify mentions of technologies, tools, and methodologies from within the system's curated skill database.

This automated parsing capability is significant for several practical reasons. It eliminates the tedious manual process of self-reporting skills one by one, which is prone to both omission and overestimation. Instead, the system objectively identifies skills that the user has explicitly mentioned in a professional context — their resume — and uses that as the foundation for the initial gap analysis. The result is an immediate, data-backed starting point that users find both credible and motivating, as it provides an honest picture of their current professional standing within seconds of completing the wizard.

6.2 Dynamic Skill Gap Analysis

The Gap Analysis Engine is the analytical core of the entire platform. Its sophistication lies in the fact that it does not merely check whether a skill is present or absent in a user's profile — a binary assessment that would be insufficient for professional evaluation. Instead, it evaluates the depth of a user's proficiency in each required skill relative to the threshold mandated by the target role. For example, a user may have rudimentary familiarity with JavaScript, which the system records as a proficiency of 30/100. If the Full Stack Developer role requires a proficiency of 75/100 for JavaScript, the engine identifies this as a significant gap rather than a satisfied requirement.

This nuanced, gradient-based assessment produces a Readiness Score that accurately reflects the user's overall proximity to professional readiness. The score is accompanied by a categorized gap report that labels each deficiency by severity (High, Medium, or Low priority), enabling users to allocate their finite learning time strategically to the skills that will yield the greatest improvement in their readiness score.

6.3 Personalized Learning Roadmaps

Based on the output of the gap analysis, the system automatically constructs a personalized, step-by-step learning roadmap tailored to the individual user. High-priority skills are surfaced first in the roadmap, and each skill is accompanied by a curated collection of learning resources — including links to official documentation, tutorial platforms, and project-based learning exercises — selected by platform administrators for their quality and pedagogical effectiveness. This curation is a significant value-add over generalist search engines, where students are often overwhelmed by the sheer volume of unverified and inconsistently quality-controlled content.

The roadmap is not static; it updates dynamically as the user progresses through their learning activities. As proficiency scores rise and gaps close, the roadmap re-prioritizes, surfacing the next most critical skills in sequence. This adaptive quality ensures that the system remains relevant and actionable throughout the user's entire preparation period.

6.4 Progress and Streak Tracking

Consistency of effort is one of the most critical determinants of learning outcomes, yet it is also one of the most challenging behaviors to sustain without external reinforcement. The platform addresses this through a comprehensive progress and streak tracking system. A learning streak is maintained for each user and increments each day that the user completes at least one learning activity. Visual indicators on the dashboard display the current streak prominently, creating a psychological incentive for users to maintain their momentum.

In addition to streak tracking, the dashboard presents an animated readiness gauge that updates in real time as new proficiency data is recorded. Historical progress charts allow users to visualize their improvement trajectory over time, providing evidence of their growth and reinforcing a sense of achievement. This gamification of the learning experience is particularly effective for student populations, for whom visible milestones and tangible progress metrics serve as powerful motivational tools.

6.5 Admin Management Portal

The Admin Management Portal ensures that the platform's knowledge base remains perpetually current and representative of evolving industry standards. Administrators are provided with a dedicated interface through which they can add new career roles, define the required skills and proficiency thresholds for each role, update existing skill definitions, and curate the library of associated learning resources. This administrative capability is essential for the long-term viability of the platform, as the technology landscape changes rapidly — frameworks rise and fall in popularity, new tools emerge, and required proficiency levels shift in response to market demand. A platform that cannot be updated would quickly become obsolete; the Admin Portal ensures that the system remains a reliable and relevant guide for students at all times.

7. ROLES AND RESPONSIBILITIES

The Skill Gap Analysis and Learning Recommendation System defines two distinct user roles, each with a specific set of permissions, capabilities, and responsibilities within the platform ecosystem. The clear delineation of roles is essential for maintaining data integrity, ensuring appropriate access control, and providing each category of user with an interface tailored to their specific needs.

7.1 User (Student)

The student is the primary and most numerous category of user on the platform. Students interact with the system to understand their current professional readiness, identify skill deficiencies, and follow a structured path toward closing those gaps. The student role is designed to be empowering, presenting complex analytical data in an accessible and actionable format without requiring any technical understanding of the underlying system.

Upon registration, a student creates a personal profile that includes their full name, email address, and optional social links to professional profiles on platforms such as GitHub and LinkedIn. The student then proceeds through the Career Wizard to upload their resume and select their target career roles. Once onboarded, the student's primary responsibilities include keeping their skill profile up to date, actively engaging with the recommended learning resources, and marking completed activities as finished to maintain the accuracy of their progress records. Students are also responsible for maintaining their learning streak by engaging with the platform on a regular, ideally daily, basis.

The student can access a comprehensive personal dashboard at any time to review their current Readiness Score, view a breakdown of identified skill gaps, navigate their personalized learning roadmap, and track their historical progress. All data visible to a student is strictly scoped to their own profile; students cannot view or modify the data of other users. This privacy boundary is enforced at the API level through JWT-based authentication and route-level authorization middleware.

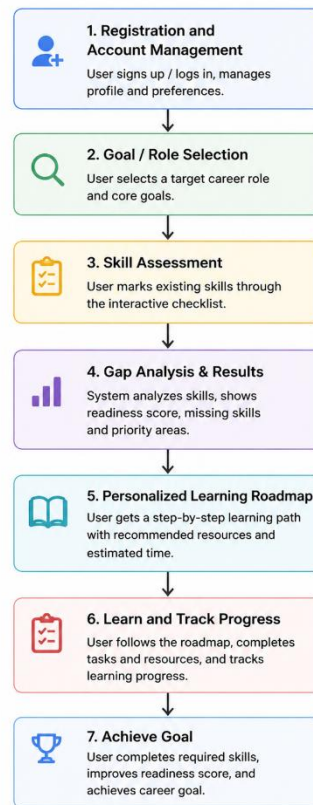
7.2 Administrator

The administrator is a privileged user whose primary responsibility is the curation and maintenance of the platform's knowledge base. Unlike students, who are consumers of the system's analytical output, administrators are producers of the data that makes that output possible. Without diligent administrative management, the role requirements and associated resources within the system would gradually become outdated, undermining the credibility and practical utility of the gap analysis for students.

An administrator logs into the platform using credentials to which the 'admin' role flag has been assigned at the database level. Once authenticated, the administrator gains access to the Admin Dashboard, which presents a high-level overview of system statistics, including the total number of registered users, available career roles, and catalogued skills. From this dashboard, the administrator can navigate to dedicated management sections for roles, skills, and learning resources.

In the Roles management section, the administrator can create new career roles, specifying the required skills and their corresponding proficiency thresholds. In the Skills management section, the administrator can add new skills to the global database, categorize them by domain, assign difficulty ratings, and estimate learning durations. In the Resources management section, the administrator curates the library of learning materials for each skill, evaluating and linking high-quality external content to ensure that students are directed to the most effective available resources. The administrator does not have the ability to view or modify individual student profile data, maintaining a clear ethical and operational separation between administrative and student-facing functions.

8. USER FLOW



8.1 Student User Flow

The student interacts with the platform through a structured flow of steps, progressing from account creation to achieving their career goal.

1. Registration and Account Management

The student signs up or logs in using their email and password. The system securely authenticates the user and initializes their profile.

2. Goal / Role Selection

The student selects a target career role (e.g., Frontend Developer) from the available options. This defines the required skills and roadmap.

3. Skill Assessment

The student marks their existing skills using an interactive checklist or resume upload. The system records their current skill levels.

4. Gap Analysis and Results

The system compares the student's skills with role requirements. It calculates the readiness score and identifies missing or weak skills.

5. Personalized Learning Roadmap

Based on the gaps, the system generates a structured learning path with recommended resources and estimated completion time.

6. Learning and Progress Tracking

The student follows the roadmap, completes tasks and resources, and tracks progress. Skill proficiency and progress data are updated dynamically.

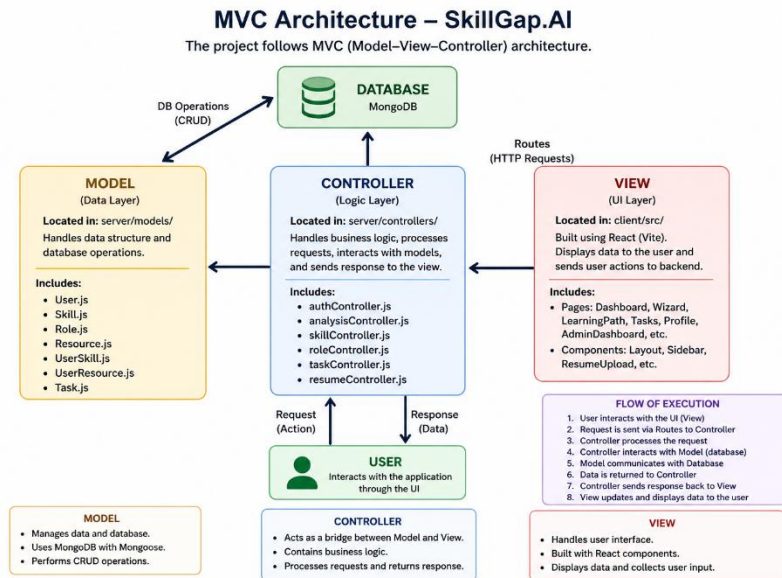
7. Goal Achievement

As the student improves their skills, the readiness score increases. Once the required level is reached, the student achieves their career goal.

8.2 Administrator User Flow

The administrator's workflow is centered around knowledge base management and platform governance. Upon logging in with administrator credentials, the admin is directed to the Admin Dashboard, which displays aggregate statistics on platform usage. The administrator's primary workflow involves navigating to the relevant management section — Skills, Roles, or Resources — and performing the necessary create, read, update, or delete (CRUD) operations. For example, to introduce a new career role into the system, the administrator navigates to the Roles section, completes the role creation form with a title and description, selects the required skills from the global database, assigns proficiency thresholds for each, and saves the entry. The new role is immediately available to all registered students. Similarly, when a new technology gains industry prominence, the administrator can add it as a new skill and associate curated resources with it, ensuring the platform evolves in step with the industry landscape.

9. MVC PATTERN



9.1 The Model Layer

The Model layer is responsible for defining the data structures of the application and for managing all interactions with the underlying database. In this project, the Model layer resides in the server/models directory of the backend codebase. Each Mongoose schema defined within this directory — User, Role, Skill, UserSkill, and Resource — constitutes a Model. These schemas specify the fields, data types, validation rules, and constraints for each document collection in the MongoDB database.

Beyond simple field definitions, the Model layer also encapsulates data-related logic through Mongoose middleware. For example, the User model includes a pre-save hook that automatically hashes the user's password using Bcrypt.js before the document is written to the database, ensuring that this security-critical operation is always performed consistently, regardless of which part of the application creates or updates a user record. Similarly, the User model exposes an instance method for password comparison that is invoked during the login process.

9.2 The View Layer

In a traditional server-rendered web application, the View layer consists of HTML templates that the server populates with data before sending them to the client. In the context of this MERN stack application, the View layer is represented entirely by the React.js frontend. Each React component is responsible for rendering a specific portion of the user interface and presenting data received from the backend in a structured, visually meaningful format. Components such as SkillCard, ProgressBar, ReadinessGauge,...

9.3 The Controller Layer

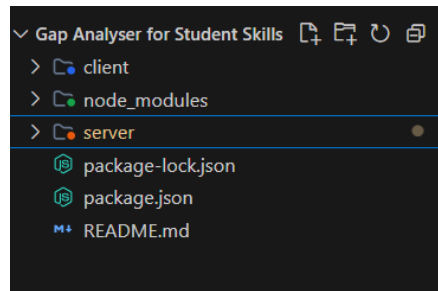
The Controller layer acts as the orchestrator of the application's business logic. Residing in the server/controllers directory, controllers receive incoming HTTP requests from the frontend (via Express.js routes), invoke the appropriate Model operations to retrieve or update data, execute business logic calculations (such as the gap analysis scoring algorithm), and construct the HTTP response that is returned to the client. Controllers are deliberately kept free of both data schema definitions (the domain of Models) and UI rendering logic (the domain of Views), ensuring they remain focused exclusively on processing.

For example, the analysisController.js file contains the analyzeGap function, which implements the core readiness score calculation. This function fetches the role requirements and the user's skill inventory from the database, performs a structured comparison, computes the weighted readiness percentage, and assembles a prioritized gap report.

9.4 Advantages of MVC in this Project

The adoption of the MVC pattern in this project yields several concrete advantages. First, it enables parallel development: the frontend development team can build and test View components independently of the backend, provided that the API contract (the shape of the request and response data) is agreed upon in advance. Second, it facilitates targeted testing: each layer can be unit-tested in isolation. Controller functions can be tested by providing mock request objects; Model schemas can be tested with an in-memory MongoDB instance. Third, it simplifies future maintenance: if the database technology were to change from MongoDB to PostgreSQL, only the Model layer would require rewriting — the Controller and View layers would remain unchanged. This modularity significantly reduces the risk and cost of future system evolution.

10. PROJECT SETUP AND CONFIGURATION



Client Setup (Frontend – React + Vite)”

- Open the **client** folder in the VS Code terminal.
- Create the React application using Vite:

```
npm create vite@latest client -- --template react
```

- Select the following options:
 - Framework: React
 - Variant: JavaScript
- Navigate to the client folder:

```
cd client
```

- Install all required dependencies:

```
npm install
```

- Start the frontend development server:

```
npm run dev
```

Server Setup (Backend – Node.js + Express):

- Open the **server** folder in the VS Code terminal.
- Initialize the Node.js project:

```
npm init -y
```

- Install required backend dependencies:

```
npm install express mongoose dotenv cors jsonwebtoken bcryptjs
```

- Create the main entry file:
 - **index.js**

Backend Folder Structure:

Inside the **server** folder, create the following structure:

Folders:

- **models/** → Contains database schemas (User, Skill, Role, Task, Resource, etc.)
- **controllers/** → Handles business logic (auth, analysis, skills, roles, tasks)
- **routes/** → Defines API endpoints and connects them to controllers
- **middleware/** → Contains authentication and request-processing middleware
- **utils/** → Helper functions and reusable logic

Additional Files:

- **.env** → Stores environment variables (MongoDB URI, JWT secret, port)
- **seed.js / seed_pro.js** → Scripts to populate database with initial data
- **package.json** → Project dependencies and scripts

Root Project Structure:

At the root level:

- **client/** → Frontend application (React + Vite)
- **server/** → Backend application (Node.js + Express)
- **package.json** → Root-level dependencies (if used)
- **README.md** → Project documentation

Running the Full Project:

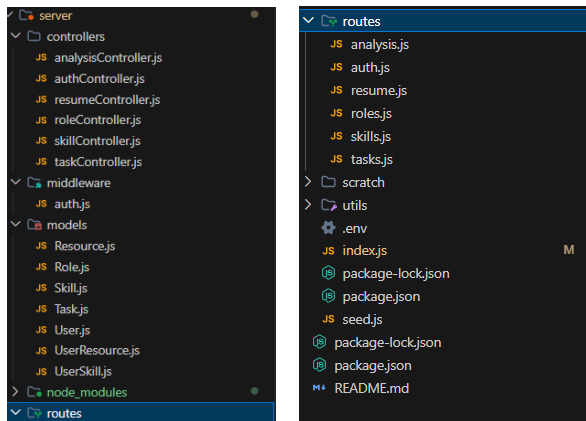
Step 1: Start Backend

```
cd server
node index.js
```

Step 2: Start Frontend:

```
cd client
npm run dev
```

11. BACKEND DEVELOPMENT



Controllers:

Handles all business logic and core functionality of the application.

- **analysisController.js**
Handles skill gap analysis, readiness calculation, and recommendation logic.
- **authController.js**
Manages user authentication including registration, login, and token handling.
- **resumeController.js**
Processes resume-related features such as extracting or analyzing user skills (if implemented).
- **roleController.js**
Manages job roles and their associated required skills.
- **skillController.js**
Handles operations related to skills such as fetching, adding, or managing skill data.
- **taskController.js**
Manages user tasks such as learning tasks, progress tracking, and completion status.

Middleware:

Contains reusable middleware functions used across routes.

- **auth.js**
Handles authentication and authorization using JWT. Protects private routes and verifies user identity.

Models:

Defines the database schemas and structure using Mongoose.

- **User.js**
Stores user account details such as name, email, password, role, and career goals.
- **Skill.js**
Represents individual skills (e.g., HTML, CSS, React).
- **Role.js**
Defines job roles and their required skill sets.
- **Resource.js**
Stores learning resources such as courses, videos, and articles linked to specific skills.
- **UserSkill.js**
Maps users to their skills and proficiency levels.
- **Task.js**
Represents user learning tasks and progress tracking.
- **UserResource.js**
Tracks resources assigned to or used by users.

Routes:

Defines API endpoints and connects them to controllers.

- **analysis.js**
Routes for performing gap analysis and generating recommendations.
- **auth.js**
Routes for user authentication (login, register).
- **resume.js**
Routes for resume upload and processing.
- **roles.js**
Routes for fetching and managing job roles.
- **skills.js**
Routes for retrieving and managing skills.
- **tasks.js**
Routes for handling user tasks and progress updates.

Utils:

Contains helper functions and reusable utilities used across the backend.

Configuration Files

- **.env**
Stores environment variables such as database URI, JWT secret, and port.
- **index.js**
Entry point of the backend server. Initializes Express app, connects to MongoDB, and sets up routes and middleware.
- **package.json**
Contains project metadata, dependencies, and scripts.
- **seed.js / seed_pro.js**
Scripts used to populate the database with initial data such as skills, roles, and resources.

The backend follows a modular architecture:

- Controllers handle logic
- Models define data structure
- Routes manage API endpoints
- Middleware secures and processes requests

This structure ensures scalability, maintainability, and clean separation of concerns.

12. DATABASE DEVELOPMENT

- **Resource.js** → Schema/model for storing learning resources (videos, articles, courses, documentation) linked to specific skills.

```
server > models > JS Resource.js > ...
1  const mongoose = require('mongoose');
2
3  const resourceSchema = new mongoose.Schema({
4    skill: {
5      type: mongoose.Schema.ObjectId,
6      ref: 'Skill',
7      required: true
8    },
9    title: {
10     type: String,
11     required: true
12   },
13   url: {
14     type: String,
15     required: true
16   },
17   type: {
18     type: String,
19     enum: ['Video', 'Article', 'Course', 'Documentation'],
20     required: true
21   },
22   level: {
23     type: String,
24     enum: ['Beginner', 'Intermediate', 'Advanced', 'Project'],
25     required: true
26   },
27   duration: {
28     type: String, // e.g., "2 hours", "4 weeks"
29     required: true
30   },
31   createdAt: {
32     type: Date,
33     default: Date.now
34   }
35 });
36
37 module.exports = mongoose.model('Resource', resourceSchema);
38 Ctrl+L for Agent
```

- **Role.js** → Schema/model for representing job roles along with required skills, proficiency levels, and industry insights.

```
server > models > JS Role.js > ...
1  const mongoose = require('mongoose');
2
3  const roleSchema = new mongoose.Schema({
4    title: {
5      type: String,
6      required: [true, 'Please add a role title'],
7      unique: true,
8      trim: true
9    },
10   description: {
11     type: String,
12     required: [true, 'Please add a description']
13   },
14   requiredSkills: [{
15     skill: {
16       type: mongoose.Schema.ObjectId,
17       ref: 'Skill',
18       required: true
19     },
20     levelRequired: {
21       type: Number,
22       default: 70 // 0-100 proficiency
23     }
24   }],
25   industryImpact: {
26     demand: { type: String, enum: ['Extreme', 'Critical', 'Very High', 'High', 'Medium', 'Low'], default: 'High' },
27     avgSalary: { type: String },
28     relevanceScore: { type: Number, min: 0, max: 100, default: 85 }
29   },
30   createdAt: {
31     type: Date,
32     default: Date.now
33   }
34 });
35
36 module.exports = mongoose.model('Role', roleSchema);
37
```

- **Skill.js** → Schema/model for defining individual skills with category, difficulty level, importance, and estimated learning time.

```
server > models > JS Skills > ...
1  const mongoose = require('mongoose');
2
3  const skillSchema = new mongoose.Schema({
4    name: {
5      type: String,
6      required: [true, 'Please add a skill name'],
7      unique: true,
8      trim: true
9    },
10   category: {
11     type: String,
12     required: [true, 'Please add a category'],
13     enum: ['Frontend', 'Backend', 'DevOps', 'Design', 'Mobile', 'Data Science', 'AI', 'Cybersecurity', 'Database', 'API', 'General'],
14   },
15   difficulty: {
16     type: String,
17     required: true,
18     enum: ['Easy', 'Medium', 'Hard']
19   },
20   importance: {
21     type: String,
22     required: true,
23     enum: ['High', 'Medium', 'Low']
24   },
25   description: {
26     type: String,
27     maxlength: [500, 'Description cannot be more than 500 characters']
28   },
29   estimatedHours: {
30     type: Number,
31     default: 10
32   },
33   createdAt: {
34     type: Date,
35     default: Date.now
36   }
37 });
38
39 module.exports = mongoose.model('Skill', skillSchema);
40
```

- **Task.js** → Schema/model for managing user tasks such as learning activities, deadlines, and completion status.

```
server > models > JS Taskjs > ...
1  const mongoose = require('mongoose');
2
3  const taskSchema = new mongoose.Schema({
4    user: {
5      type: mongoose.Schema.ObjectId,
6      ref: 'User',
7      required: true
8    },
9    title: {
10     type: String,
11     required: [true, 'Please add a title']
12   },
13   description: {
14     type: String
15   },
16   deadline: {
17     type: Date,
18     required: [true, 'Please add a deadline']
19   },
20   status: {
21     type: String,
22     enum: ['Pending', 'Completed'],
23     default: 'Pending'
24   },
25   createdAt: {
26     type: Date,
27     default: Date.now
28   }
29 });
30
31 module.exports = mongoose.model('Task', taskSchema);
32
```

- **User.js** → Schema/model for storing user account details, authentication data, career goals, skills, projects, and progress tracking.

```
server > models > JS User.js > ...
1 const mongoose = require('mongoose');
2 const bcrypt = require('bcryptjs');
3
4 const userSchema = new mongoose.Schema({
5   name: {
6     type: String,
7     required: [true, 'Please add a name'],
8   },
9   email: {
10    type: String,
11    required: [true, 'Please add an email'],
12    unique: true,
13    match: [
14      /^([a-z][\w-]*@([a-z][\w-]*)([a-z]{2,3})?)/$/i,
15      'Please add a valid email'
16    ]
17  },
18  password: {
19    type: String,
20    required: [true, 'Please add a password'],
21    minlength: 6,
22    select: false
23  },
24  role: {
25    type: String,
26    enum: ['user', 'admin'],
27    default: 'user'
28  },
29  targetRole: {
30    type: mongoose.Schema.ObjectId,
31    ref: 'Role'
32  },
33  targetRoles: [{
34    type: mongoose.Schema.ObjectId,
35    ref: 'Role'
36  }],
37  dob: Date,
38  mobile: String,
39  socials: {
40    linkedin: String,
41    github: String,
42    portfolio: String,
43    twitter: String
44  },
45  skills: [String],
46  experience: [{
47    company: String,
48    role: String,
49    duration: String,
50    description: String
51  }],
52  projects: [{
53    title: String,
54    description: String,
55    link: String,
56    technologies: [String]
57  }],
58  streak: {
59    current: { type: Number, default: 0 },
60    lastActive: { type: Date, default: Date.now },
61    longest: { type: Number, default: 0 }
62  },
63  createdAt: {
64    type: Date,
65    default: Date.now
66  }
67 });
68
69 // Encrypt password using bcrypt
70 userSchema.pre('save', async function() {
71   if (!this.isModified('password')) {
72     return;
73   }
74   console.log('Hashing password...');
75   const salt = await bcrypt.genSalt(10);
76   this.password = await bcrypt.hash(this.password, salt);
77   console.log('Password hashed.');
```

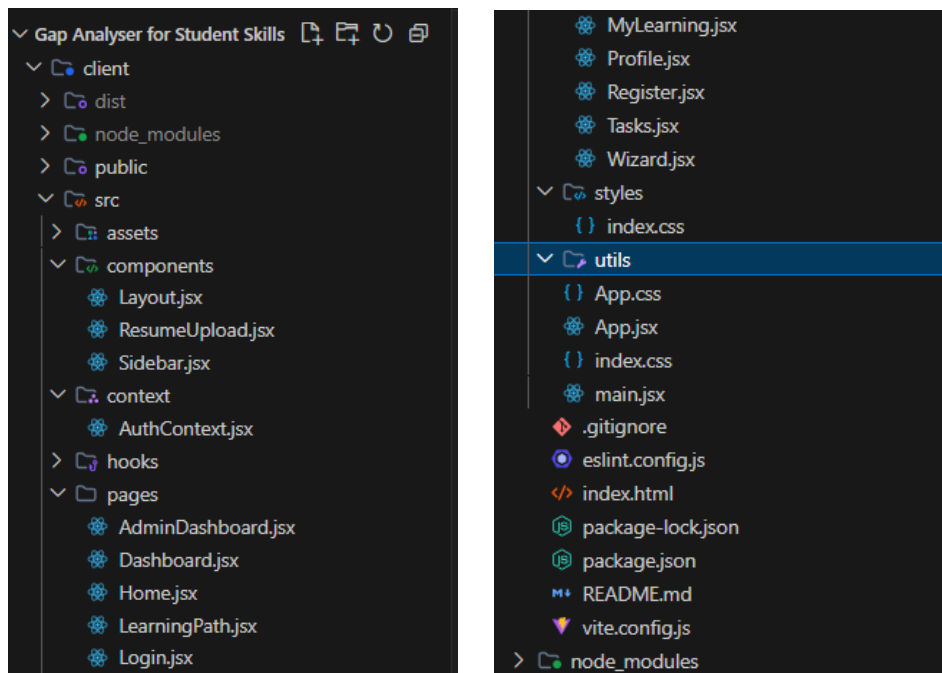
- **UserResource.js** → Schema/model for tracking resources assigned to users and marking them as completed.

```
server > models > JS UserResource.js > ...
1 const mongoose = require('mongoose');
2
3 const userResourceSchema = new mongoose.Schema({
4   user: {
5     type: mongoose.Schema.ObjectId,
6     ref: 'User',
7     required: true
8   },
9   resource: {
10    type: mongoose.Schema.ObjectId,
11    ref: 'Resource',
12    required: true
13  },
14  completedAt: {
15    type: Date,
16    default: Date.now
17  }
18 });
19
20 // Ensure a user can only mark a resource as completed once
21 userResourceSchema.index({ user: 1, resource: 1 }, { unique: true });
22
23 module.exports = mongoose.model('UserResource', userResourceSchema);
24
```


- **UserSkill.js** → Schema/model for mapping users to their skills, including proficiency level and learning status.

```
server > models > JS UserSkill.js > ...
1  const mongoose = require('mongoose');
2
3  const userSkillSchema = new mongoose.Schema({
4    user: {
5      type: mongoose.Schema.ObjectId,
6      ref: 'User',
7      required: true
8    },
9    skill: {
10     type: mongoose.Schema.ObjectId,
11     ref: 'Skill',
12     required: true
13   },
14   proficiency: {
15     type: Number,
16     default: 0,
17     min: 0,
18     max: 100
19   },
20   status: {
21     type: String,
22     enum: ['To Learn', 'Learning', 'Mastered'], ➔ Tab to Jump
23     default: 'To Learn'
24   },
25   lastUpdated: {
26     type: Date,
27     default: Date.now
28   }
29 });
30
31 // Compound index to ensure unique skill per user
32 userSkillSchema.index({ user: 1, skill: 1 }, { unique: true });
33
34 module.exports = mongoose.model('UserSkill', userSkillSchema);
35
```

13. FRONT-END DEVELOPMENT



Components:

- **Layout.jsx** → Main layout component that structures the overall UI including sidebar and content area.
- **Sidebar.jsx** → Navigation component for accessing different sections like dashboard, roadmap, learning, and tasks.
- **ResumeUpload.jsx** → Component for uploading and processing user resumes.

Context:

- **AuthContext.jsx** → Manages authentication state, user session, and global access to user data.

Hooks:

- Custom React hooks for handling reusable logic and state management across components.

Pages:

- **Home.jsx** → Landing page introducing the platform and guiding users to get started.
- **Login.jsx** → User authentication page for logging into the system.
- **Register.jsx** → Registration page for creating new user accounts.
- **Dashboard.jsx** → Main user dashboard displaying readiness score, skills, and analytics.
- **AdminDashboard.jsx** → Admin panel for managing users, roles, skills, and system data.
- **Wizard.jsx** → Roadmap wizard for selecting target role and marking known skills.
- **LearningPath.jsx** → Displays structured learning roadmap and recommended resources.
- **MyLearning.jsx** → Tracks user learning progress and completed resources.
- **Tasks.jsx** → Manages user tasks, deadlines, and completion status.
- **Profile.jsx** → Displays and manages user profile information.

Styles:

- **index.css** → Global styling file defining overall theme, layout, and UI consistency.

Root files:

- **App.jsx** → Main application component handling routing and layout integration.
- **main.jsx** → Entry point of the React application, rendering the app to the DOM.
- **App.css** → Application-specific styling.

Github Link:

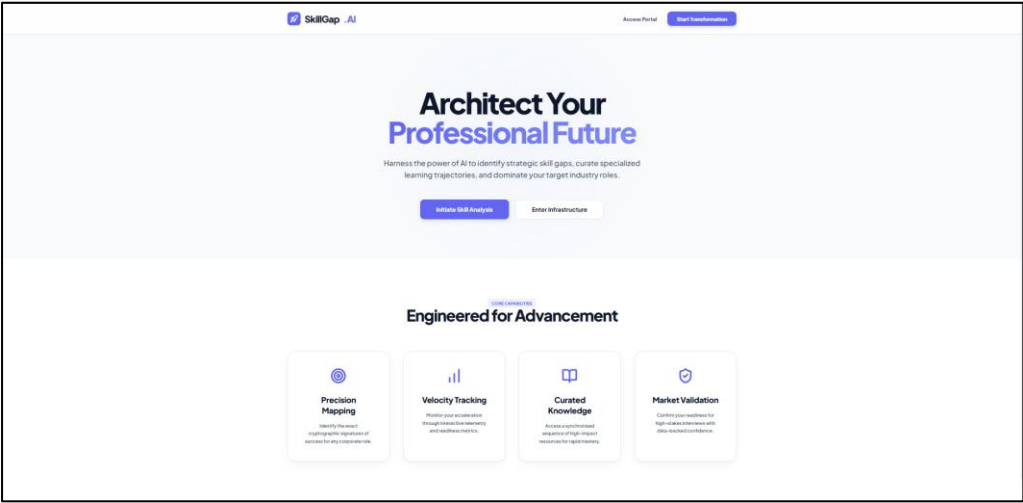
<https://github.com/mohammedafthabrazahussain/Skill-Gap-Analysis>

Code Explanation:

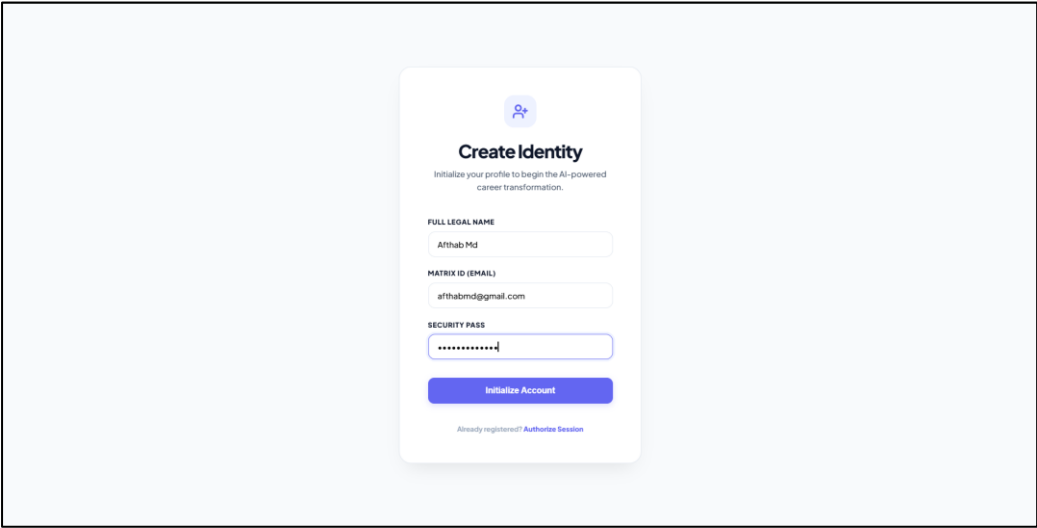
[https://drive.google.com/file/d/1OqEcmm0hUn5UEHAr4DqG5Aegpvz7lokZ/view?usp=drive link](https://drive.google.com/file/d/1OqEcmm0hUn5UEHAr4DqG5Aegpvz7lokZ/view?usp=drive_link)

14. APPENDIX:

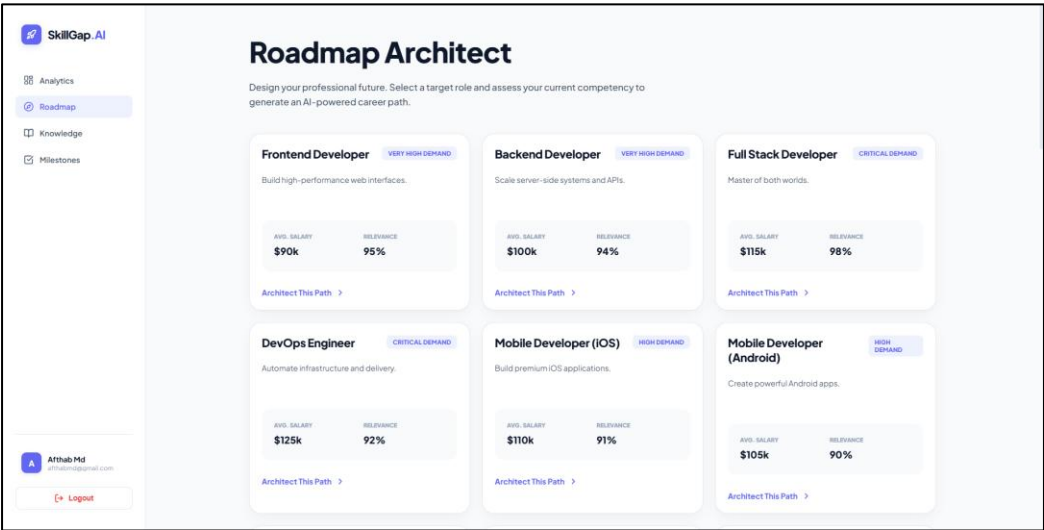
- Homepage



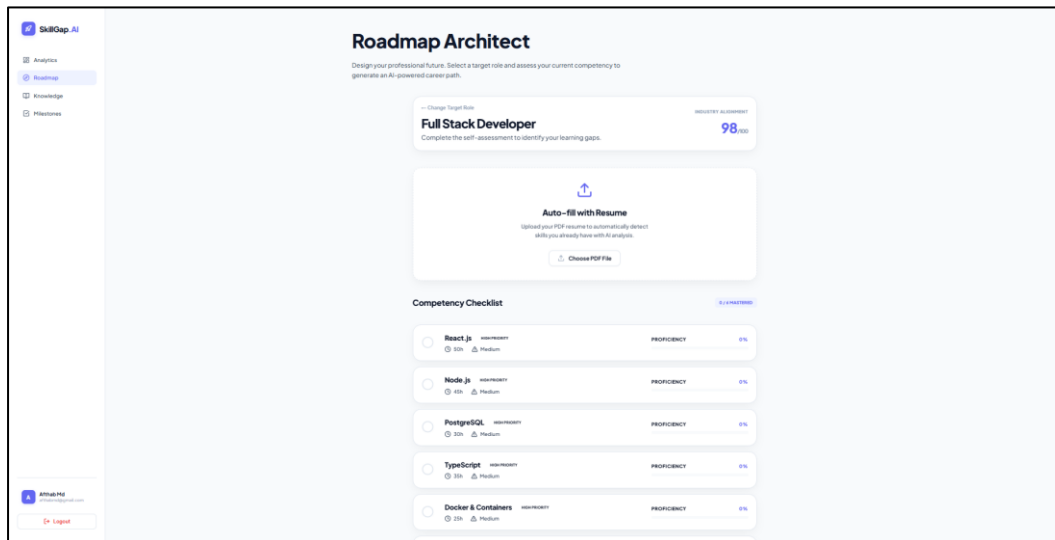
- Signup Page



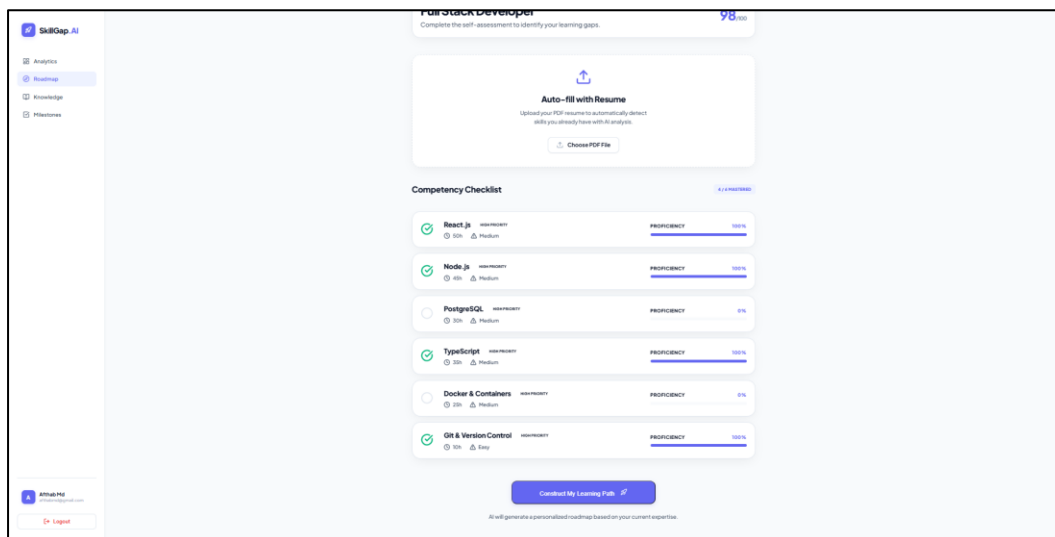
- Roadmap (user Selects his interested role)



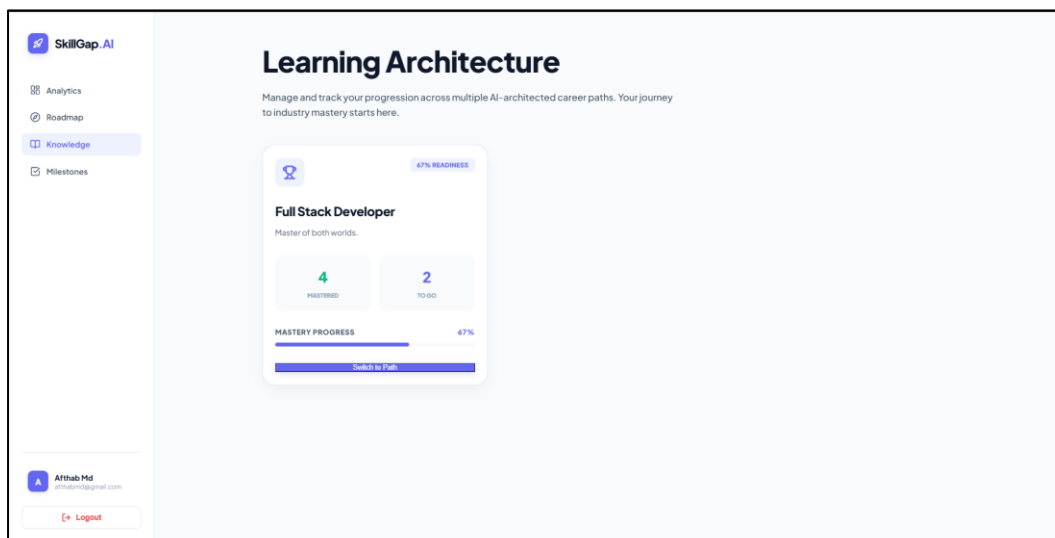
- Roadmap Architect, where user selects and describes his known Skills and proficiency.



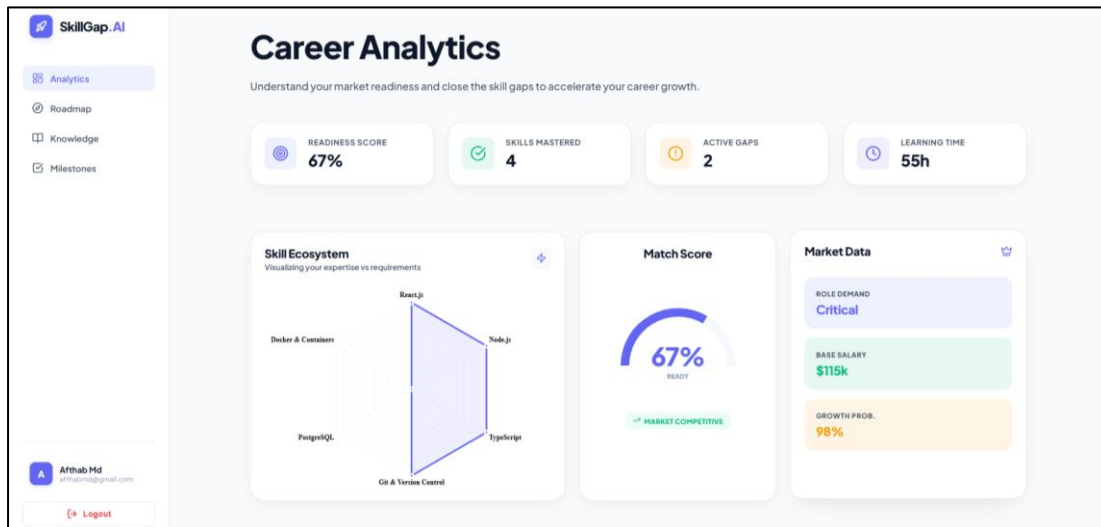
- Selects the Skills and rates the Proficiency and creates the analytics



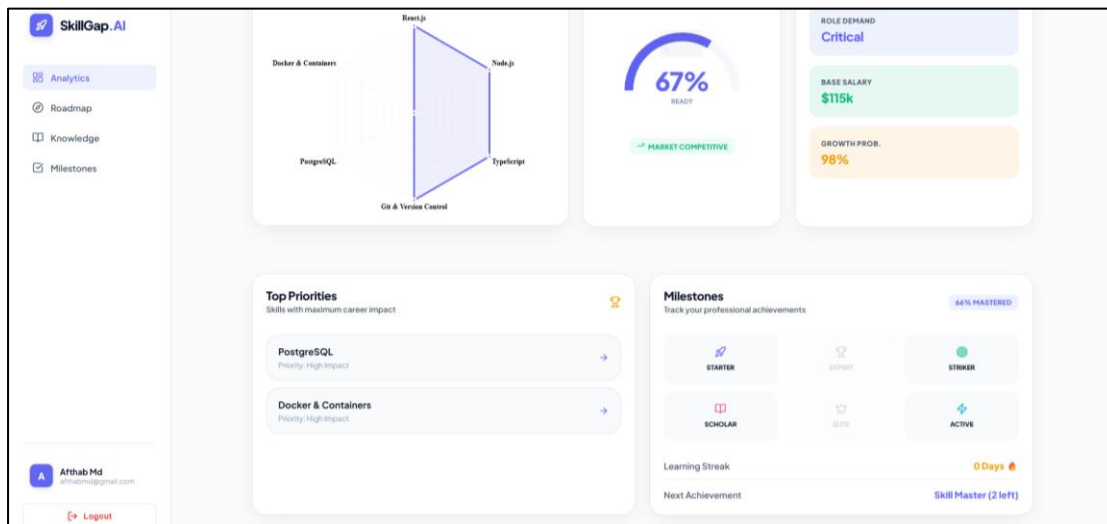
- In Knowledge page it stores the users interested ongoing or completion course



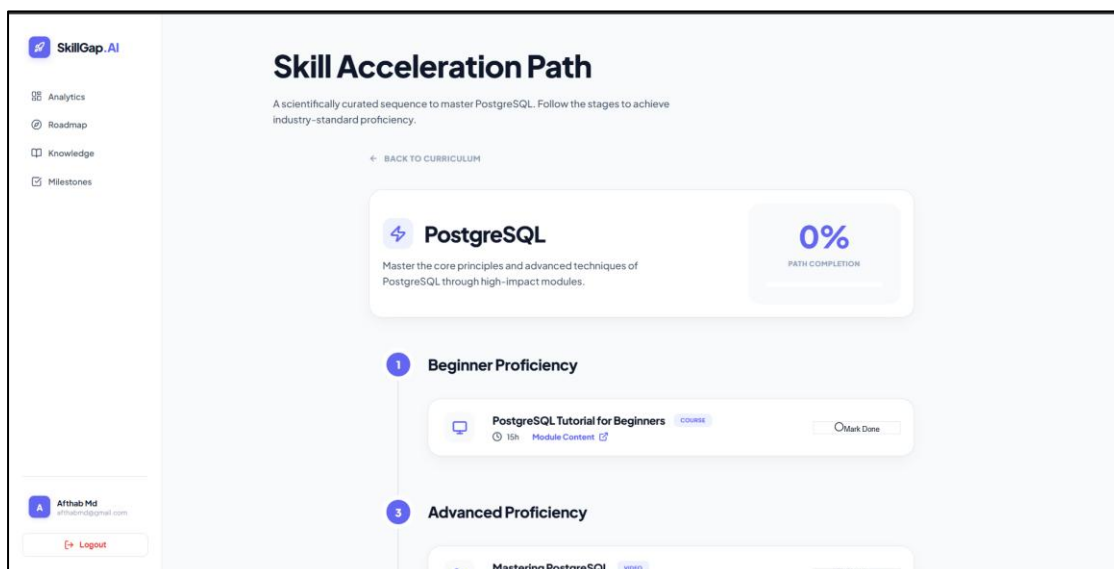
- Analytics page where user can track his progress and chosen role's Duration



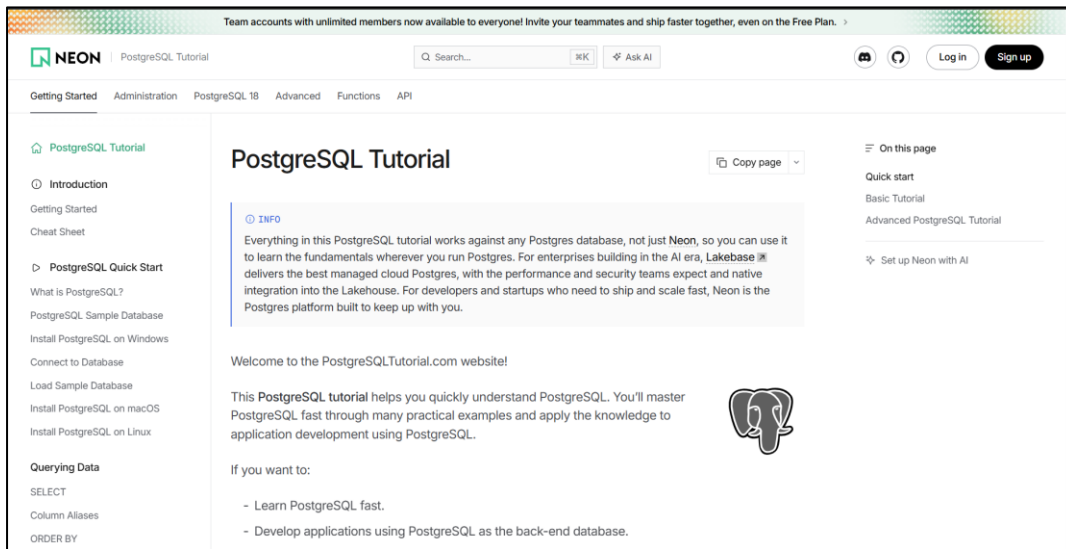
- Here, Analytics shows the next steps to complete and the milestones of him in the website



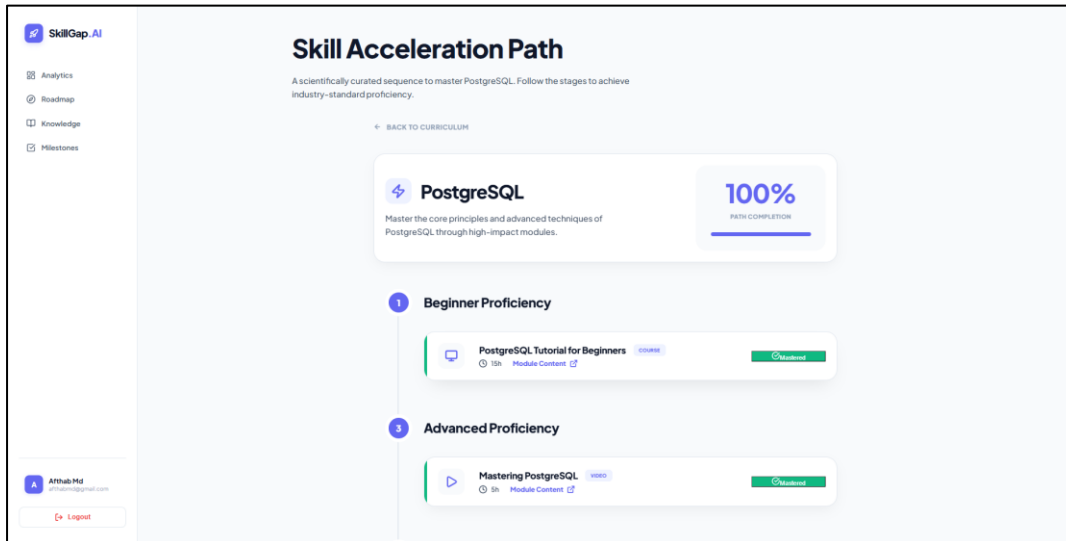
- Analytics Provides the resources to the user to complete and gain knowledge



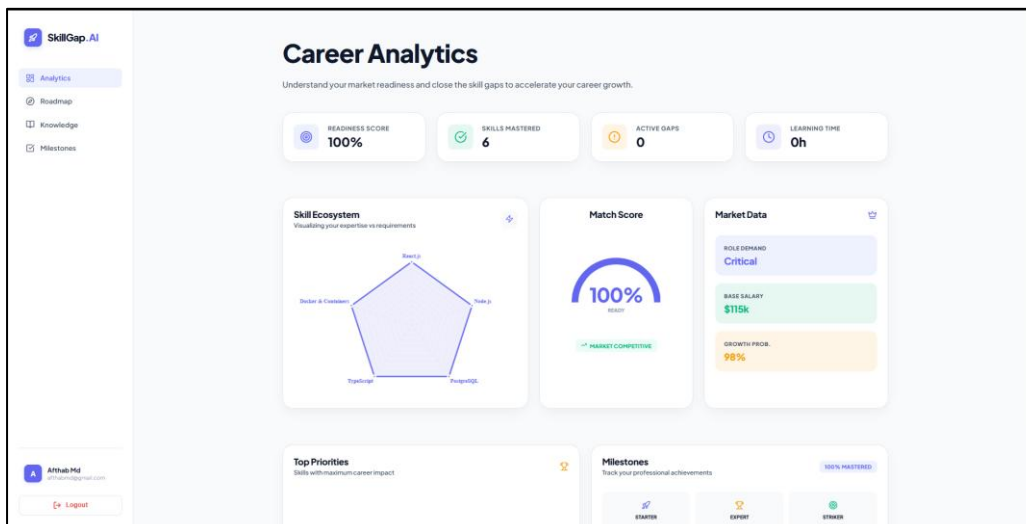
- It Directs to the resources for the user to learn



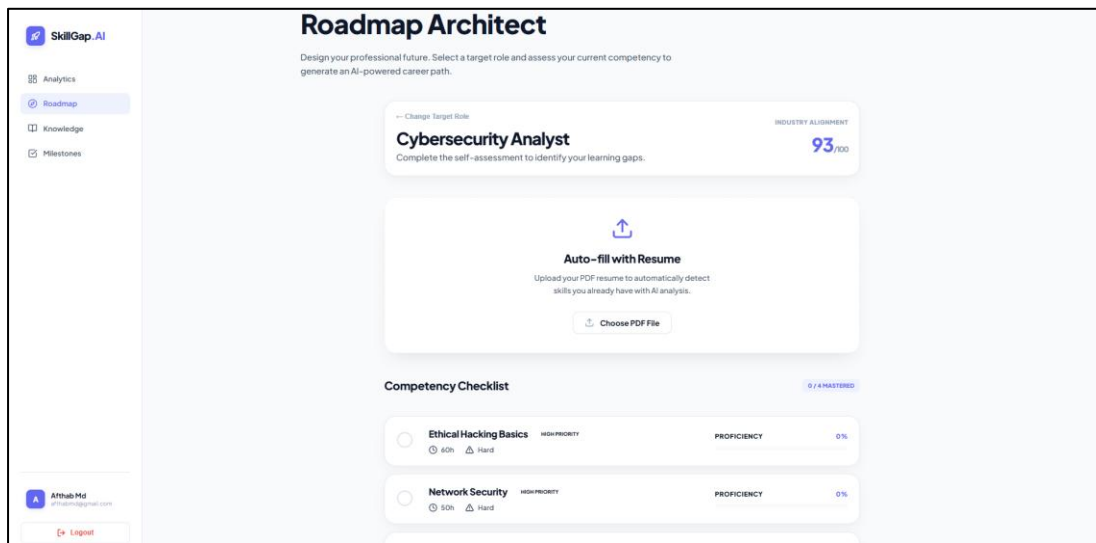
- After completion of that Skill it will be marked as completed



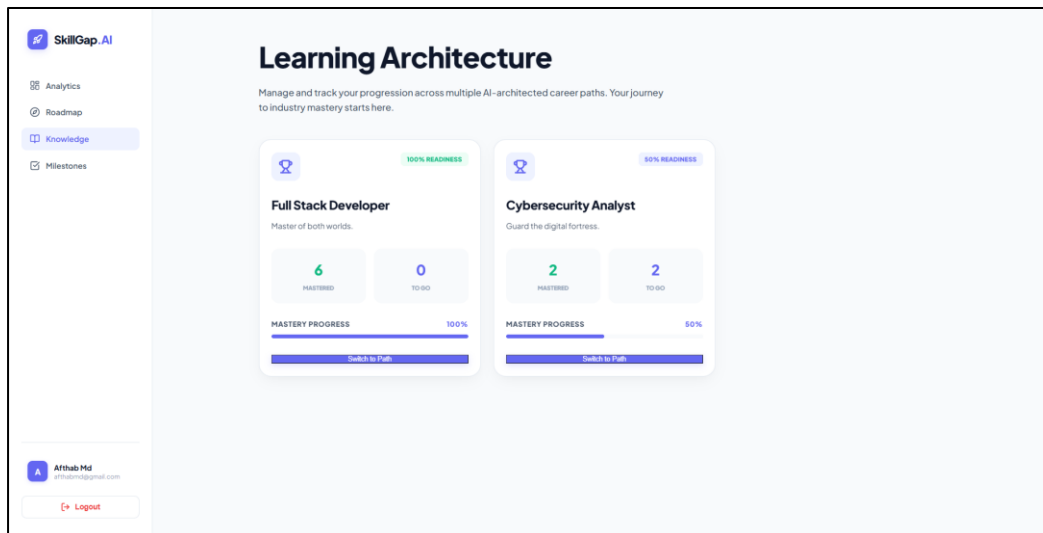
- Now the Users Analytics dashboard will be updated and shows his progress



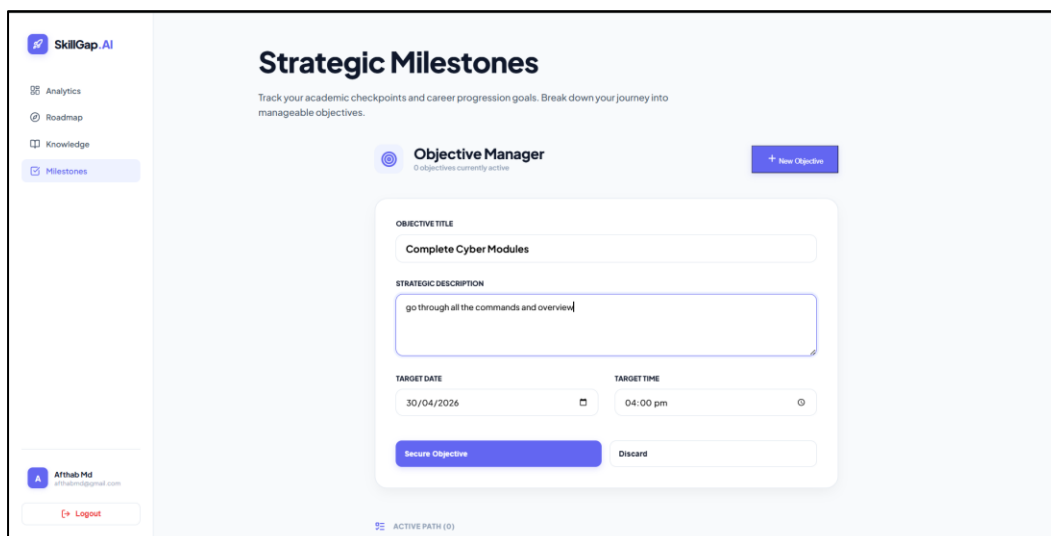
- User can also learn/develops his skills in Multiple Roles



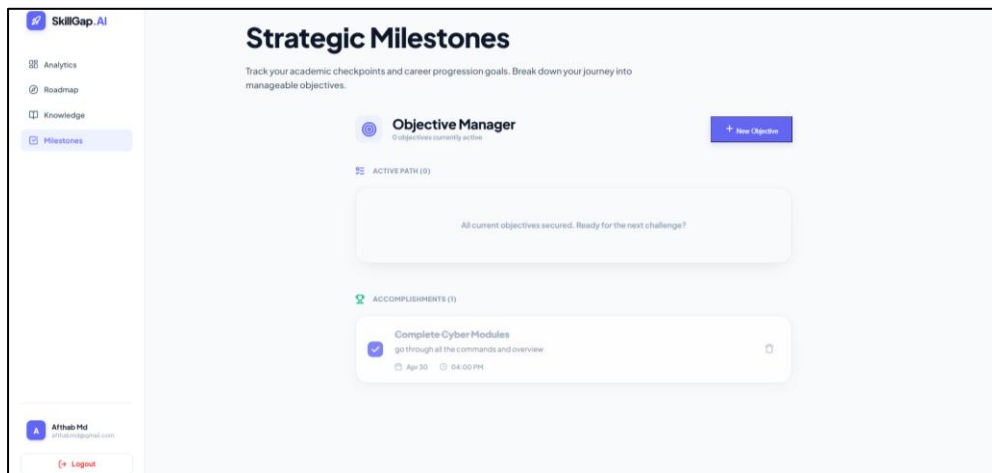
- So, In the knowledge page it will track the users selected/learning roles



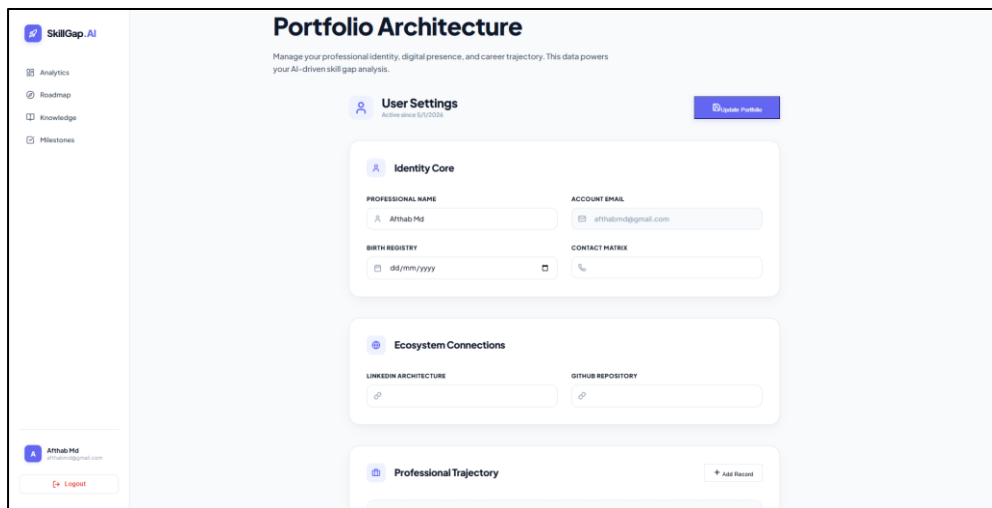
- Milestones page – In this page User can enter their tasks with deadlines and completion time.



- After completing their tasking it will be marked as done



- User Profile, where they can edit their details



- All the Users, Roles, Skills, tasks, Resources, Recommendations, etc.. are stored in MongoDB

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total Index size
gapanalyses	-	32.77 kB	113 kB	5	224.00 B	1	32.77 kB
jobroles	-	32.77 kB	578.00 B	2	289.00 B	2	65.54 kB
learningresources	-	36.86 kB	974.00 B	4	243.00 B	1	36.86 kB
recommendations	-	20.48 kB	9.76 kB	33	295.00 B	1	20.48 kB
resources	-	36.86 kB	13.53 kB	62	218.00 B	1	32.77 kB
roles	-	32.77 kB	13.06 kB	25	522.00 B	2	65.54 kB
skills	-	36.86 kB	8.75 kB	54	161.00 B	2	73.73 kB
tasks	-	36.86 kB	384.00 B	2	192.00 B	1	36.86 kB
userresources	-	24.58 kB	368.00 B	4	92.00 B	2	49.15 kB
users	-	32.77 kB	5.38 kB	19	283.00 B	2	65.54 kB
userskills	-	32.77 kB	2.03 kB	16	127.00 B	2	65.54 kB

Project Demonstration:

https://drive.google.com/file/d/1HwYzOatAgDkw6QngSZQWN48PJZrxJdkS/view?usp=drive_link

CONCLUSION

The Skill Gap Analysis and Learning Recommendation System represents a holistic and technically robust solution to one of the most pressing challenges facing engineering students today: the disconnect between academic preparation and industry expectations. By integrating automated resume parsing, intelligent gap analysis, personalized learning roadmaps, and gamified progress tracking within a unified, full-stack web platform, the system empowers students to take a data-driven approach to their career development.

Built on the MERN technology stack with adherence to the MVC architectural pattern, the system is modular, scalable, and maintainable — qualities that make it not only a valuable tool for students today but also a strong foundation for future enhancements. Potential extensions of the system include integration with real-time job postings APIs to dynamically update role requirements, AI-powered learning resource recommendations using collaborative filtering, and peer comparison features that allow students to benchmark their readiness against anonymized cohort data.

This project conclusively demonstrates that technology, when thoughtfully applied to the domain of education and career development, can serve as a powerful equalizer — providing every student, regardless of institutional support or socioeconomic background, with the structured guidance of a world-class career mentor. The Skill Gap Analysis and Learning Recommendation System stands as a meaningful contribution to the intersection of education technology and software engineering.

REFERENCES

1. MongoDB, Inc. (2024). MongoDB Documentation. <https://www.mongodb.com/docs/>
2. Facebook, Inc. (2024). React Documentation. <https://react.dev/>
3. OpenJS Foundation. (2024). Node.js Documentation. <https://nodejs.org/docs/>
4. Express.js Foundation. (2024). Express.js Documentation. <https://expressjs.com/>
5. Mongoose ODM. (2024). Mongoose Documentation. <https://mongoosejs.com/docs/>
6. Auth0 Blog. (2023). Introduction to JSON Web Tokens. <https://jwt.io/introduction/>
7. Chart.js Foundation. (2024). Chart.js Documentation. <https://www.chartjs.org/docs/>
8. Framer Motion. (2024). Framer Motion API Reference. <https://www.framer.com/motion/>
9. Vite. (2024). Vite Official Documentation. <https://vitejs.dev/guide/>
10. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.